

Advanced Programming Course Project

Common code and Protocols. v0.2

A.Y. 2025/2026

This document describes the GALAXY initialization file, common code and protocols involved in the course project of the 2025/2026 A.Y.

The goals of this document are:

- to guide the creation of common code and document it;
- to detail the protocols used by different groups, so that EXPLORERS can communicate with PLANETS even if they are developed by different people.

The document and its contents are not final, and are in fact expected to be improved upon time. You can modify the definition and behavior of the various objects, as well as adding new objects and removing them.

1 Galaxy initialization file

The GALAXY initialization file contains one line per PLANET. Each line contains a `planet_id` followed by list of PLANETS that are the ids of its neighboring PLANETS. Connections between PLANETS are bidirectional. Each line must not contain repetitions.

Example Line `w1 w3 w4 w5` means that PLANET `w1` is connected with PLANETS `w3`, `w4` and `w5`.

2 Common Crate

This section provides the code of the common crate in its initial version. The code comprises the definition of structs, enums and other objects.

Each object must be accompanied by a detailed documentation of its purpose and behavior; that is, its **specification**. For example, this documentation must state whether an object can or cannot be constructed from within a certain component.

2.1 The Sunray struct

```
1 pub struct Sunray;
```



This is *underspecified*: it does not state that a sunray object can only be constructed by the orchestrator. The absence of this statement would allow a group to construct a sunray object at will.

2.2 The Asteroid struct

```
1 struct Asteroid;
2
3 impl Asteroid {
4     fn deflect_with_rocket(self, rocket: Rocket) {}
5 }
```



The requirement is that an asteroid can be deflected with a rocket, entailing that a group should discard a rocket to do so:

- Does the method effectively capture the requirement, or is it useless?
- What's the alternative which balances correctness and ergonomics the most?

2.3 The Energy Cell struct

```
1 struct EnergyCell { charge: i32 }
2
3 impl EnergyCell {
4     pub fn charge(sunray: Sunray) {
5         self.charge += 1;
6     }
7 }
```



There are bugs: for example, there is no check that the charge exceeds the allowed maximum. Furthermore, it is underspecified: does it have other methods?

2.4 The Rocket struct

```
1 struct Rocket;
2
3 impl Rocket {
4     fn new(energy_cells: Vec<EnergyCell>) -> Self { /* TODO */ }
5 }
```



Underspecification:

- There is no explanation of how to build a Rocket.
- What is the behavior of a Rocket?
- Should it have other methods? Should it have a `fire` method, against an Asteroid? Or should the Asteroid have a method to call on PLANETS, which then will defend themselves in case they have Rockets?

2.5 The Planet object

There can only be four kinds of PLANETS to can be instantiated.



- Does it make sense to introduce a struct or a trait for PLANET? Could those allow to enforce the above requirement?
- Should you create four structs, one per planet, and a factory?

2.6 The Resource struct

A resource is a struct that cannot be cloned or copied. It must implement a `Resource` trait that extends the `Display` and has a method for converting it to a `&'static str`.

To combine resources, the `Add` and `From Rust` traits could suffice. Here we provide another possible base implementation to follow.

```
1 struct Water; // a basic resource
2 struct Mud; // a complex resource
3
```

```

4 pub trait Combine<Lhs, Rhs> {
5     fn combine(lhs: Lhs, rhs: Rhs) -> Self;
6 }
7
8 impl Combine<Earth, Water> for Mud {
9     fn combine(lhs: Earth, rhs: Water) -> Mud {
10         Mud
11     }
12 }

```

Example usage:

```

1 fn main() {
2     let earth = Earth;
3     let water = Water;
4     let mud = Mud::combine(earth, water);
5 }

```



The first document states that there should be an enum for all types of resources, basic resources and complex resources; those are clearly missing here. Furthermore, this is underspecified: it does not describe the variants of the enums.

3 Protocols

There are three protocols in this project:

- the orchestrator-PLANET protocol, which describes how the orchestrator communicates with each planet (and vice-versa);
- the orchestrator-explorer protocol, which describes how the orchestrator communicates with each explorer (and vice-versa);
- the explorer-planet protocol, which describes how each planet communicates with explorers (and vice-versa).

These protocols describe which messages can be sent from one entity to the other, and what kind of reply is expected from said messages.

3.1 Orchestrator-Planet protocol

Reception of these messages must be logged.



Precisely, how is logging done? What library does it employ, and how should log statements be formatted?

```

pub enum OrchestratorToPlanet { Sunray, Asteroid, StartPlanetAI, StopPlanetAI }

pub enum PlanetToOrchestrator {
    SunrayAck { planet_id: u32 },
    AsteroidAck { planet_id: u32, destroyed: bool },
    StartWorldAiResult { planet_id: u32, timestamp: u32 },
    StopWorldAiResult { planet_id: u32, timestamp: u32 }
}

```

3.1.1 Sunrays

The **Sunray** message is followed by a **SunrayAck** message indicating the planet id that received the sunray.



Precisely, which components can send this message?

3.1.2 Asteroids

The `Asteroid` message is followed by a `AsteroidAck` message indicating the planet id that received the sunray.



Underspecification: what does the `destroyed` field mean?

3.1.3 Starting and stopping the Planet AI



Does the `timestamp` field serve any purpose?

3.2 Orchestrator-explorer protocol

Reception of these messages must be logged.



Not all the following enum variants are properly documented! This must be fixed!

```
pub enum OrchestratorToExplorer {
    ResetExplorerAI,
    MovedToPlanet { sender_to_new_planet: mpsc::Sender<ExplorerToPlanet> },
    /// Requests the explorer the id of the planet in which it is currently hosted.
    CurrentPlanetRequest,
    /// Requests the explorer to request the planet in which it is currently hosted
    /// the resource it supports to generate.
    CurrentPlanetResponse { planet_id: Option<u32> },
    SupportedResourceRequest,
    /// Requests the explorer to request the planet in which it is currently hosted
    /// the combination of resources it supports.
    SupportedCombinationRequest,
    GenerateResourceRequest,
    CombineResourceRequest { first: &'static str, second: &'static str },
}

pub enum explorerToOrchestrator {
    StartExplorerAIResult { ... },
    StopExplorerAIResult { ... },
    MovedToWorldResult { ... },
    SupportedResourceResponse { ... },
    SupportedCombinationResponse { ... },
    GenerateResourceResponse { resources: Vec<String> },
    CombineResourceResponse { },
}
```



This is *underspecified*: it does not state what is the sequence of messages. If a certain message is sent, what are the possible replies? How does the sequence of messages evolve?

3.2.1 Starting and stopping the explorer AI

The explorer AI can be started by using sending a `ResetExplorerAI` message. The explorer must send a `StartExplorerAIResult` message back as an acknowledgment.

3.2.2 Moving the explorer to another planet

The explorer can be moved to another planet by sending a `MovedToPlanet` message to it. This message contains a sender to the new planet that the explorer can use to send messages to it. The explorer must send a `MovedToWorldResult` message back as an acknowledgment.



Underspecifications:

- What happens if the attached sender is the same of the one currently owned by the explorer?
- What happens if this message is received while the explorer AI is active?

3.2.3 Generating and combining resources

The `GenerateResourceRequest` message can be used to request the explorer to request planet in which it is currently hosted to generate a basic resource that is supported. Likewise, the `CombineResourceRequest` message can be used to request the explorer to request the planet in which it is currently hosted to combine two resources. These message have corresponding responses.



Underspecifications:

- What are the `first` and `second` fields?

3.3 Explorer-Planet protocol

Reception of these messages must be logged.

```
pub enum explorerToWorld {
    /// Requests the explorer to request the planet in which it is currently hosted
    /// the resource it supports to generate.
    SupportedResourceRequest,
    /// Requests the explorer to request the planet in which it is currently hosted
    /// the combination of resources it supports.
    SupportedCombinationRequest,
    /// Requests the explorer to request the planet in which it is currently hosted
    /// to generate a basic resource that is supported.
    GenerateResourceRequest,
    /// Requests the explorer to request the planet in which it is currently hosted
    /// to combine two resources.
    CombineResourceRequest { first: &'static str, second: &'static str },
    /// Requests how many energy cells the planet has, and their current charge.
    EnergyCellRequest,
}

pub enum planetToExplorer {
    SupportedResourceResponse { ... },
    SupportedCombinationResponse { ... },
    GenerateResourceResponse { ... },
    CombineResourceResponse { ... },
    EnergyCellResponse { number: std::num::NonZeroUsize },
}
```